

TP MapReduce

Dans ce TP, nous manipulerons l'exemple « **Word Count** » ou « Compteur d'occurrence de mots » vu en cours.

Préparation du TD

1. Démarrer la machine virtuelle.

2. Démarrer les services de Hadoop avec les commandes :

```
$ start-dfs.sh
```

```
$ start-arn.sh
```

3. Vérifier le bon fonctionnement de Hadoop. Utiliser l'outil jps pour lister les processus Java en cours d'exécution :

```
$ jps
```

```
3631 Jps
```

```
3336 NodeManager
```

```
3212 ResourceManager
```

```
3651 NameNode
```

```
2863 DataNode
```

```
3052 SecondaryNameNode
```

4. Vérifier que l'environnement de compilation est fonctionnel.

(Vérifier que Java est bien installé sur votre machine « `java -version` »).

5. Décompresser l'archive :

```
$ unzip tp_bigdata_mr.zip
```

Manipulation de l'exemple

Nous allons maintenant compiler le code d'exemple Java du cours (compteur d'occurrence de mots).

1. Changer le répertoire courant :

```
$ cd tp_bigdata_mr
```

2. Compiler le programme Hadoop (le tout sur la même ligne) :

```
javac -classpath "/home/ghazouani/Desktop/Hadoop/hadoop-2.7.3/share/hadoop/common/hadoop-common-2.7.3.jar:/home/ghazouani/Desktop/Hadoop/hadoop-2.7.3/share/hadoop/mapreduce/hadoop-mapreduce-client-common-2.7.3.jar:/home/ghazouani/Desktop/Hadoop/hadoop-2.7.3/share/hadoop/common/lib/commons-cli-1.2.jar:/home/ghazouani/Desktop/Hadoop/hadoop-2.7.3/share/hadoop/mapreduce/hadoop-mapreduce-client-core-2.7.3.jar" WCount*.java
```

La compilation va générer trois fichiers .class: un pour chacune de nos classes (driver, map et reduce).

3. On va désormais packager le programme d'exemple au sein d'un fichier .jar. Créer l'arborescence liée au nom du package avec la commande :

```
$ mkdir wordcount
```

Et déplacer, par la suite, les fichiers compilés au sein de cette arborescence : `$ mv *.class wordcount`

4. Générer le .jar :

```
$ jar -cvf wcount.jar -C . wordcount
```

5. En préparation de l'exécution de notre programme Hadoop, nous allons maintenant déplacer le texte du poème sur HDFS. Exécuter la commande :

```
$ hadoop fs -put poeme.txt /
```

et vérifier sa présence avec la commande : `$ hadoop fs -ls /`

6. Enfin, exécuter notre programme Hadoop avec la commande :

```
$ hadoop jar wcount.jar wordcount.WCount /poeme.txt /results
```

La commande devrait prendre tout au plus quelques secondes à s'exécuter. Si tout s'est passé correctement, un message « mapreduce.Job: map 100% reduce 100% » devrait s'afficher.

7. Vérifier la présence des fichiers de résultats dans le répertoire /results avec la commande :

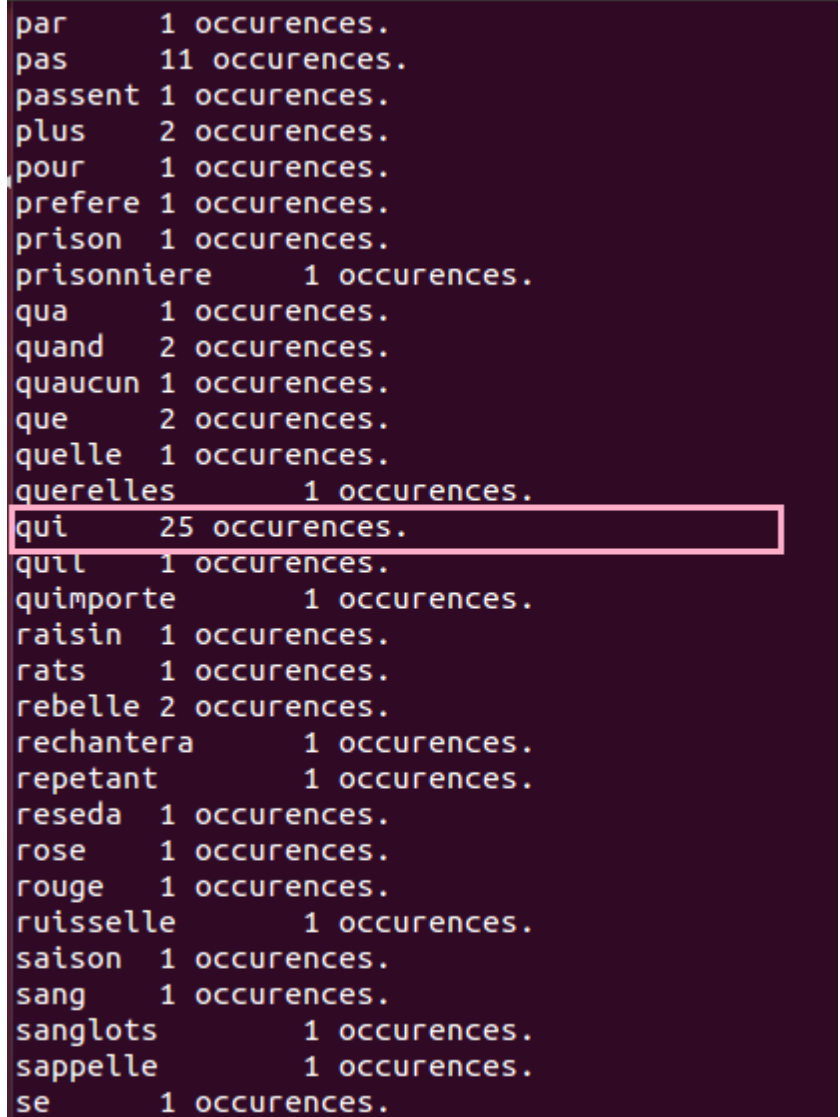
```
$ hadoop fs -ls /results
```

(un fichier `_SUCCESS` devrait être présent, ainsi qu'un fichier `part-r-00000`).

8. Enfin, afficher les résultats finaux avec la commande :

```
$ hadoop fs -cat /results/part-r-00000
```

Le mot « qui » devrait être le plus présent au sein de la poème (25 occurrences). Consultez la figure 1 ci-dessous.



```
par      1 occurrences.  
pas      11 occurrences.  
passent  1 occurrences.  
plus     2 occurrences.  
pour     1 occurrences.  
prefere  1 occurrences.  
prison   1 occurrences.  
prisonniere 1 occurrences.  
qua      1 occurrences.  
quand    2 occurrences.  
quacun   1 occurrences.  
que      2 occurrences.  
quelle   1 occurrences.  
querelles 1 occurrences.  
qui      25 occurrences.  
quit     1 occurrences.  
quimporte 1 occurrences.  
raisin   1 occurrences.  
rats     1 occurrences.  
rebelle  2 occurrences.  
rechantera 1 occurrences.  
repetant 1 occurrences.  
reseda   1 occurrences.  
rose     1 occurrences.  
rouge    1 occurrences.  
ruisselle 1 occurrences.  
saison   1 occurrences.  
sang     1 occurrences.  
sanglots 1 occurrences.  
sappelle 1 occurrences.  
se       1 occurrences.
```